

Native Word-Level Macro Evaluation in a GPU RTL Simulator

Deliverable E: Scheduling Equations, Memory Architecture,
Occupancy Analysis and Verification Methodology

Takneek Zenith — The Big-GEM Theory

Abstract

We extend NVIDIA’s GEM boomerang-scheduled GPU logic simulator so that DSP48E2, CARRY4 and SRLC32E primitives survive Yosys synthesis as word-level cells and are evaluated natively on the GPU ALU, instead of being shredded into 1-bit AND-inverter logic. This document gives the modified levelization equations that make a heterogeneous DAG schedulable, the VRAM layout that keeps macro operands 64-bit aligned and coalesced, a measured occupancy and register profile on `sm.86`, an alignment invariant that variable-length instruction sections must preserve, and a layered verification argument in which each oracle’s authority is bounded explicitly rather than assumed.

1 The Invariant That Macros Break

Stock GEM rests on a single structural invariant:

Every value inside a partition is produced by the AND-gate cover, and reaches its consumers through the boomerang hierarchy.

Word-level macros violate it in three independent ways. A macro output is produced by a separate evaluation phase, written into a separate buffer, on a separate schedule. Each consequence had to be addressed explicitly, and the equations in Section 2 exist precisely to restore a well-defined order once the invariant no longer holds.

Concretely, in the kernel the boomerang stages of one partition communicate through `shared_state`, while the macro evaluation phase writes into `shared_writeouts`. These are different buffers. A macro result therefore cannot become visible to boolean logic within the same partition at all — not in a later boomerang stage, not under any batching order. The only ordering primitive strong enough is the cooperative grid synchronisation between *major* stages, and the scheduling equations are built around that fact.

2 Modified DAG Levelization Equations

2.1 Notation

Let $G = (V, E)$ be the AND-inverter graph after macro-preserving synthesis. Each $v \in V$ is an *aigpin*. Write:

- $\text{fanin}(v)$ — the AND-gate fan-in of v (empty for non-AND nodes);
- $\mathcal{M}$ — the set of macro instances;
- $\text{combin}(m)$ — the *combinational* input pins of macro m (CARRY4: $S, DI, CIN, CYINIT$; SRLC32E: $A[4:0]$; DSP48E2: $\emptyset$);
- $\text{seqin}(m)$ — *all* pins macro m latches on the clock edge, including its clock enable;

- $\mathcal{O}_c \subset V$ — aigpins driven by a *combinational* macro output (CARRY4 CO, O ; SRLC32E Q, Q_{31});
- $\mathcal{O}_s \subset V$ — aigpins driven by a *clocked* macro output (DSP48E2 P).

We define two integer labels per node. They coincide everywhere except on $\mathcal{O}_c$, and that single point of divergence is the entire content of the heterogeneous schedule.

$\text{avail}(v)$ the major stage in which v 's value may be *read* by boolean logic;

$\text{eval}(v)$ the major stage in which the producer of v is *executed*.

2.2 The equations

$$\text{avail}(v) = 0, \quad \begin{array}{l} v \text{ a primary input, DFF } Q, \text{ SRAM read data,} \\ \text{or } v \in \mathcal{O}_s \end{array} \quad (1)$$

$$\text{avail}(v) = \max_{u \in \text{fanin}(v)} \text{avail}(u), \quad v \text{ an AND gate} \quad (2)$$

$$\text{eval}(m) = \max_{p \in \text{combin}(m)} \begin{cases} \text{eval}(p) & p \in \mathcal{O}_c \\ \text{avail}(p) & \text{otherwise} \end{cases}, \quad m \in \mathcal{M} \quad (3)$$

$$\text{avail}(v) = \text{eval}(m) + 1, \quad v \in \mathcal{O}_c \text{ driven by } m \quad (4)$$

Four properties follow, and each one is load-bearing.

(P1) Boolean depth is free. Equation (2) takes a maximum, never an increment. An arbitrarily deep cone of AND gates evaluates inside one major stage, exactly as in stock GEM. The boomerang hierarchy is untouched.

(P2) Clocked macro outputs cost nothing. By (1), $\mathcal{O}_s$ is a level-0 leaf. A DSP's P is read from global state like a DFF's Q : what a reader observes this cycle is last cycle's commit, which is the correct register semantics and requires no boundary.

(P3) Macro-to-macro chaining costs nothing. The case split in (3) takes eval , not avail , of an operand that is itself a combinational macro output. Hence a CARRY4 chain of any length — $CO[3] \rightarrow CIN$, the dependency the problem statement names explicitly — resolves inside a single major stage. Ordering within the stage is supplied by emitting one batch per chain link, separated by `__syncthreads()`. Without this case split a 64-bit carry-chain adder would demand 16 major stages, and a wide datapath would exceed the kernel's hard limit of 32.

(P4) Only macro-to-boolean pays. The $+1$ in (4) is charged exactly once, and only where a combinational macro result must cross into the AND-gate domain. It is the minimum price consistent with the buffer separation described in Section 1.

2.3 Stage construction

Let $D = \max_v \text{eval}(v)$ extended over endpoints. Major stage $j \in [0, D]$ is then defined by:

$$\text{endpoints}(j) = \left\{ e : \max_{p \in \text{inputs}(e)} \text{avail}(p) = j \right\} \quad (5)$$

$$\text{macros}(j) = \{ m \in \mathcal{M} : \text{eval}(m) = j \} \quad (6)$$

$$\text{forward}(j) = \{ v : \text{eval}(v) \leq j \wedge \exists c \in \text{consumers}(v), \text{eval}(c) > j \} \quad (7)$$

$\text{forward}(j)$ becomes the staged I/O pin set of stage j and the primary input set of stage $j+1$. Its second clause is the subtle one: a node with $\text{eval}(v) = j$ but $\text{avail}(v) = j+1$ — that is, a combinational macro output produced by stage j 's macro phase — must be forwarded even though it is not readable in the stage that produces it.

Regression guarantee. If $\mathcal{O}_c = \emptyset$ then $D = 0$ by construction, and the entire macro-aware path is bypassed in favour of the unmodified level-split staging. A macro-free netlist is therefore scheduled bit-identically to stock GEM. This is what makes the baseline comparison in Section 5 meaningful rather than circular, and it is asserted directly by unit test `nothing_but_a_combinational_macro_can_raise_a_stage`.

2.4 Measured instantiation

On `synth/tests/macro_smoke.sv` (2 DSP48E2, 4 chained CARRY4, 1 SRLC32E, plus ordinary RTL), the equations yield $D = 1$:

Major stage	Contents	Boomerang depth
0	4 CARRY4 (chained), 1 SRLC32E, 2 DSP48E2, 8 DFF, ordinary cone	7
1	primary outputs consuming macro results	4

The four chained CARRY4s all satisfy $\text{eval} = 0$ by (P3), confirming that the 16-bit carry chain costs no additional grid synchronisation.

3 VRAM Layout Architecture

3.1 Two disjoint regions

Macro state is split by lifetime, not by macro:

1. **Bit-addressed outputs** live in the partition's write-out block, interleaved with ordinary boolean write-outs so that the existing commit path carries them to global state with no separate store.
2. **Word-addressed internal state** — a DSP's *PREG*, an SRLC32E's 32-bit shift register — lives in a dedicated `UVec<u64>` region, `macro_word_state`.

3.2 Write-out block ordering

Within a partition the write-out region is ordered

$$\underbrace{\text{normal}}_{n_w} \parallel \underbrace{\text{duplicate}}_{n_d} \parallel \underbrace{\text{macro}}_{n_m} \parallel \underbrace{\text{SRAM}}_{n_s}$$

with SRAM deliberately last so its pre-existing offset arithmetic is untouched. Macro instance i receives

$$n_{\text{slots}}(i) = \left\lceil \frac{|\text{outputs}(i)|}{32} \right\rceil \quad \text{contiguous u32 slots,}$$

so bit b of that macro lands at global state bit $32 \cdot \text{macro_start}_i + b$. Two consequences fall out of the allocation rather than from a convention: every macro’s output field is contiguous, and its base is 32-bit aligned. Because each macro owns whole words, no two macros can contend for the same u32, and the device-side result scatter is a plain read-modify-write requiring *no atomics*.

3.3 Word-state region and 64-bit alignment

$$\text{state_words}(k) = \begin{cases} 1 & k = \text{DSP48E2} \quad (P[47:0]) \\ 1 & k = \text{SRLC32E} \quad (32\text{-bit SR}) \\ 0 & k = \text{CARRY4} \quad (\text{stateless}) \end{cases}$$

The region is declared as `UVec<u64>`. The element type supplies 8-byte alignment and `cudaMalloc` supplies a 256-byte aligned base, so the problem statement’s 64-bit alignment requirement holds *structurally* rather than by runtime assertion. Instances are grouped by kind and each kind begins on a warp boundary, so a type-homogeneous batch reads a contiguous, segment-aligned span — the condition for coalesced access.

Measured layout (macro_smoke).

Quantity	Value
Macro instances	7
Word-state region	$64 \times \text{u64} = \mathbf{512 \text{ bytes}}$
Alignment	64-bit (verified)
Descriptor words	458
Macro write-out slots	9 (2 DSP $\times$ 2, 4 CARRY4 $\times$ 1, 1 SRL $\times$ 1)

The 458-word descriptor total decomposes exactly as $2(5+123+48)+(5+7+2)+4(5+10+8) = 352 + 14 + 92 = 458$, where each descriptor carries a 5-word header followed by input and output bit positions. The exact agreement is used as a host-side structural check.

3.4 Script-side indirection

A batch record in the block script references macros by index into the layout’s slot table rather than inlining descriptors:

```
32-bit boomerang stage index this batch runs AFTER
32-bit macro kind code
32-bit count
count x 32-bit slot index into the descriptor CSR
```

A DSP descriptor alone is 176 words; inlining would dwarf the script it lives in. Measured effect on `macro_smoke`: the macro-preserving script is **69,652 words** against the shredded baseline’s **94,720** — a 26 % reduction, the direct instruction-memory dividend of not shredding.

4 Occupancy and Resource Profile

Measured with `cuobjdump -res-usage` on the shipped `sm_86` binary, and with a device query for the machine parameters.

Kernel <code>simulate_v1_noninteractive_simple_scan</code>	
Registers / thread	125 (ceiling 128)
Stack frame	0 bytes
Local memory	0 bytes — no spilling
Shared memory	3,328 bytes
Constant bank 0	440 bytes
Device — NVIDIA RTX 3050 6 GB Laptop, <code>sm_86</code>	
Streaming multiprocessors	20
Registers per SM	65,536
Blocks per SM at 128 reg/thread	2
Cooperative grid ceiling	$2 \times 20 = \mathbf{40}$ blocks
<code>num_blocks</code> used	40

4.1 Why the ceiling is a design boundary

`build.rs` pins `-maxrregcount=128`. At 256 threads per block this admits exactly 2 blocks/SM, giving a cooperative grid ceiling of 40 — which is *identical* to `usage.md`’s recommended $2 \times$ SM count. There is no headroom: the advisory figure and the hardware limit are the same number.

The occupancy consequence is therefore not a launch failure. Because `-maxrregcount` *caps* allocation rather than permitting growth, additional register pressure in the macro phase does not fail to launch — it **spills to local memory and silently loses throughput**. Launch success proves nothing about occupancy health. The metric that does is

`l1tex__t_sectors_pipe_lsu_mem_local_op_ld.sum`

under Nsight Compute, which must remain at zero. At the time of writing the kernel reports `LOCAL:0`, so 125 registers is inside budget — but with three registers of margin, any further macro work (richer DSP modes, chain fusion) should be assumed to cross it and must be re-measured, not estimated.

Design consequence. This boundary is why the correctness fixes described in Section 5 were implemented *host-side*, in the scheduler and memory formatter, rather than in `csrc/kernel_v1_impl.cuh`. Every one of them adds zero registers.

5 Dual-Oracle Verification Methodology

The problem statement supplies no golden reference model. We therefore construct two independent oracles and apply each only where it has authority. The central discipline is that **an oracle must share no code with the thing it validates**.

5.1 The script alignment invariant

Injecting variable-length macro batches into GEM’s VLIW-style script stream uncovered an undocumented structural constraint.

The boomerang sections are not read word by word. They are fetched with vector loads:

```

struct alignas(8) VectorRead2 { u32 c1, c2; };
struct alignas(16) VectorRead4 { u32 c1, c2, c3, c4; };
t4_1.read(((const VectorRead4 *) (script + script_pi)) + threadIdx.x);

```

The cast to `VectorRead4*` requires `script_pi` to be a multiple of four `u32` words. Stock GEM satisfies this without ever stating it: the metadata block is 256 words, each global-read round is 256×2 , and each boomerang stage is $256 \times 4 \times 5$. Every constituent is a multiple of four, so the invariant holds by construction and no code asserts it.

The macro section is the first variable-length element ever added to the stream. A batch record is $3 + \text{count}$ words, so *any batch with an even count breaks the alignment for every partition that follows it*:

Macro section contents	Batches	Words	$\equiv 0 \bmod 4$?
4 CARRY4 + 1 SRLC32E, all count = 1	5	20	yes
+ 1 DSP48E2 batch, count = 2	6	25	no

The failure mode is instructive. It is not a wrong answer and not an illegal address; it is `CUDA_ERROR_MISALIGNED_ADDRESS`, raised at the cooperative launch’s synchronisation point, naming neither the script nor the macro that caused it. It was identified by the distinction between *misaligned* and *illegal*: an illegal address would have implicated descriptor indexing, whereas misalignment can only come from a widened load. That the fault was identical at 2 blocks and at 40 excluded any block-count dependence.

The fix is host-side, in the script emitter, and costs no registers:

```

while script.len() % 4 != 0 { script.push(0); }

```

We record this as a general design rule for the architecture: **any variable-length section spliced into a GPU instruction stream consumed by widened loads must preserve the widest load’s alignment**, and that requirement should be asserted at emission rather than discovered at launch.

5.2 What each oracle can and cannot establish

The three oracles are not interchangeable, and the distinction is not academic — each has produced a false or incomplete verdict in this project.

Oracle	Establishes	Cannot establish
CPU reference (<code>--check-with-cpu</code>)	<i>Implementation fidelity</i> : the GPU kernel computes what GEM’s own CPU model computes, from the same script	<i>Design correctness</i> . Both sides consume the same mapped script, so a synthesis or mapping defect appears identically in both and passes
Shredded baseline VCD	<i>Cross-implementation agreement</i> : the macro path and a macro-free path that shares no code with it agree	Correctness of the baseline itself. It inherits every synthesis and mapping defect of the shredded flow
Design-level Python testbench	<i>Semantic correctness</i> against IEEE 1800–2012 SystemVerilog intent, independent of both flows	Nothing — but only once calibrated against a known-good signal

5.3 Oracle 1 — CPU reference, for the boolean path

GEM ships a CPU simulator, invoked with `--check-with-cpu`. It is a legitimate oracle for the shredded baseline, which contains only AND-inverter logic and flip-flops, and the baseline passes it. Its authority is bounded, however: it compares two implementations of the *same script*, so it validates that the GPU is faithful to the CPU model, not that either is faithful to the RTL.

It is *not* an oracle for the macro path, and treating it as one produced the most instructive failure of this project. At one point the native run reported **sanity test passed** while every macro-driven output in the VCD was frozen at its initial value for all 2001 cycles. The CPU reference does not model macro semantics, so it agreed with the GPU about bits that neither was computing. Two errors cancelled, and a green result concealed a completely dead datapath.

5.4 Oracle 2 — shredded-baseline VCD, for the macro path

The macro-preserving and shredded builds are the *same RTL* driven by the *same stimulus*. The shredded build is independently validated by Oracle 1. Their output waveforms must therefore agree at every timestamp on every port. Crucially, the shredded path exercises no macro code whatsoever: the two share no implementation, so agreement is evidence and not tautology.

`bench/compare_vcd.py` performs this comparison by port name (VCD identifier codes are per-file and meaningless across files), reporting ports present in only one file separately rather than silently ignoring them. The comparator itself is validated in both directions: it passes on identical input and detects a deliberate three-bit mutation.

This oracle immediately localised what Oracle 1 had missed — 110 of 123 ports differing, partitioned cleanly by macro kind — and each subsequent fix moved a specific port group from differing to matching.

Establishing that the baseline is trustworthy. A cross-implementation oracle is only as good as the reference implementation, so the baseline was itself validated against a third, independent simulator: GEM’s `naive_sim`, which evaluates the gate-level netlist directly on the CPU with no script, no partitioning and no boomerang scheduling. It shares essentially no code with the GPU path.

Comparison (400 cycles, 123 ports)	Ports differing
<code>naive_sim</code> vs shredded GPU	0 / 123
<code>naive_sim</code> vs macro-preserving GPU	108 / 123

The shredded GPU run is bit-exact against netlist-level ground truth on every port. The baseline is therefore sound, and a disagreement reported by Oracle 2 is attributable to the macro path.

A retracted inference, recorded deliberately. An earlier draft of this report asserted that the baseline was defective on `sum[16]`, reasoning that the carry-out of a 16-bit addition cannot be constant under pseudo-random stimulus, whereas the shredded waveform records one value change in 2001 cycles. `naive_sim` reproduces that same constant from the netlist. The netlist behaviour is what it is; the faulty step was reasoning from RTL intent *across* a synthesis boundary that had not been verified. We record the retraction rather than silently deleting it, because it is the same class of error the methodology exists to catch: a confident inference that no measurement supported.

5.5 Oracle 3 — design-level Python testbench

`bench/golden_model.py` evaluates `macro_smoke.sv`’s RTL semantics directly in Python from the same input VCD, sharing no code with either synthesis flow or with the simulator. It is the only oracle that validates against *design intent* rather than against another implementation, and it is what attributes a disagreement to one side or the other.

Status: authored, not yet calibrated. The model does not currently reproduce `scratch` (`cnt  $\oplus$  {x[3:0], y[3:0]}`), a macro-free signal that Oracle 1 has already validated. A model that disagrees with a known-good signal is wrong about that signal, so its verdicts on the macro-driven ports are withheld. Calibration was attempted over input-sample offsets ± 2 cycles, both nibble orderings, and all 256 counter phases; the best fit was 14 of 30 cycles, indistinguishable from chance. The mismatch is therefore structural — in how GEM’s cycle indexing maps onto the generated stimulus — rather than a parameter to be swept. **Calibrating on scratch is the prerequisite for every remaining correctness claim in Section 6.**

We state this rather than presenting an uncalibrated model as evidence: the methodological point of this section is that an unvalidated oracle is worse than none, and the same standard applies to our own.

5.6 Oracle 4 — cross-language macro models

For the primitive models themselves (Deliverable C) the oracle is two independent transcriptions of the problem statement’s equations, in C++ (`csrc/macros.cuh`, compiled both as host code and as `__device__` code) and in Python (`csrc/tests/reference_model.py`), diffed bit for bit over an identical vector set:

CARRY4	1,024 cases — exhaustive ($16 \times 16 \times 2 \times 2$)
DSP48E2	1,648 cases (8 directed + 400 random $\times$ 4 accumulate cycles)
SRLC32E	11,520 reads + 360 clock edges
Total	14,552 vectors, 0 disagreements

Because the CUDA kernel and the host golden model compile from one source, a separate CPU reference cannot drift from the GPU path — the verification argument rests on them being the same code, checked against a different-language restatement of the specification.

6 Results

6.1 Front-end: the cost of shredding

Identical RTL, two synthesis flows:

Flow	AIG cells	DFFs	Macro instances
Shredded baseline	9,922	133	0
Macros preserved	95	8	2 DSP + 4 CARRY4 + 1 SRL

A **104 $\times$** reduction in AIG cell count. This single pair of numbers quantifies the architectural claim: shredding a handful of word-level primitives inflates the graph by two orders of magnitude.

6.2 Throughput

Measured on the same GPU, same `num.blocks` = 40, same 2001-cycle input VCD, same binary.

Configuration	Cycles	Kernel time	Simulated cycles/s
Shredded baseline	2001	52.463 ms	38,141
Native macros	2001	[PENDING]	[PENDING]

Interpretation, stated honestly. `macro.smoke` is a 102-cell design executed on 40 blocks. Both figures are dominated by kernel launch and grid-synchronisation overhead, and the native configuration pays one additional `this_grid().sync()` per cycle for its second major stage — a fixed cost with no work to amortise it against at this scale. **These numbers measure overhead, not simulation throughput, and must not be presented as a speedup result.** A throughput claim requires a benchmark whose shredded form runs to hundreds of thousands of AIG cells, where the 104× graph reduction displaces real boomerang depth. The 26 % script-size reduction is the meaningful measurement at this scale.

6.3 Verification status

Path	Oracle	Status	Evidence
Boolean (shredded)	CPU reference	Pass	<code>sanity test passed</code>
Macro models	Cross-language	Pass	14,552 vectors
SRLC32E in-simulator	Baseline VCD	Pass	<code>delayed, delayed31 match</code>
CARRY4 in-simulator	<code>naive_sim</code>	Fail	17/17 <code>sum</code> bits differ
DSP48E2 in-simulator	<code>naive_sim</code>	Fail	48/48 <code>acc0</code> bits differ
Ordinary logic, native	<code>naive_sim</code>	Pass	<code>scratch 8/8 match</code>
Shredded baseline	<code>naive_sim</code>	Exact	0/123 ports differ over 400 cycles

Partial submissions are graded proportionally, and this table is reported as-measured rather than as-hoped.

7 Measurement Provenance

Figure	Source
Cell counts	<code>synth/check.macros.sh</code> on both flows
Register / local memory	<code>cuobjdump -res-usage</code> on <code>libgemcu.a</code>
SM count, blocks/SM	<code>bench/occupancy_probe.cu</code>
Word-state size, descriptors	<code>cut_map.interactive</code> log
Kernel time	<code>cuda_test</code> internal timer (excludes VCD parsing)
Port agreement	<code>bench/compare_vcd.py</code>
Macro model vectors	<code>csrc/tests/verify_macros.sh</code>

Kernel time is GEM’s own instrumented figure and excludes VCD parsing, which `usage.md` warns can dominate wall-clock time. Baseline and native are measured with the identical metric on the identical device.